MANUAL TEST GUIDE · MODULE 08 OF 11

Documentation Functional-Flow Test Walkthroughs

Scenario-driven QA guide for the **logistics.documentation** layer — the **one polymorphic** `logistics.document` envelope (HBL / MBL / MAWB / HAWB / CMR / RWB / Manifest + SI / VGM / DO / POD), the DOC-12 status lifecycle, the checklist → gate spine, versioning + final lock, and the B/L release flow.

> **How to use:** Work the scenarios top to bottom against a tenant loaded with `--with-demo=all`. Each step gives a precondition, an action, the expected result, and a ✔ **checkpoint** to tick. Green = happy path; amber/red steps deliberately trip a guard to confirm it holds.

PREPARED FOR	SOURCE INPUTS	DEMO LANE	DATE
Manual QA / UAT	logistics-documentation use-case · documentation.md · module code	Globex INMUM → SGSIN	15 July 2026

SECTION 01

01 What Documentation does & how to read this guide

Documentation is **one model**, `logistics.document` — every transport document the platform issues (HBL, MBL, MAWB, HAWB, CMR, CIM-RWB, Manifest) *and* every operational / uploaded document (SI, VGM, DO, POD, pre-alert) share the same envelope. `document_type_id` discriminates; there are **no per-type models** (BR-DOC-01). Container / seal numbers are **read-through** from equipment.usage, never re-typed (BR-DOC-03).

23	3	11	8
COMMANDS TO EXERCISE	TEST SCENARIOS	DOC-12 STATUS VALUES	DOWNSTREAM GATE POINTS

The DOC-12 lifecycle you will observe

One unified `status` enum is the single source of truth for document state (BR-DOC-STA-01). Transitions are guarded (BR-DOC-STA-02); some require a reason (BR-DOC-STA-04). The upload flow is a slice of the same lifecycle.

```
status : required → pending → draft → under_review → approved → released → completed
        offshoots: correction_required · rejected · expired · waived (terminal)
upload : uploaded → reviewed → accepted (→ approved) | rejected (→ rejected)
issue  : draft → approved  — "approved" IS the issued/final state; there is no separate "issued"
```

Two orthogonal facets set on every document

ORIGIN

- **generated** — DRE-rendered (HBL, SI, VGM, manifest…)
- **uploaded** — user file via `upload`, carries an `upload_status`

REASON-GATED & TERMINAL STATES

- **reason required** → correction_required · rejected · waived
- **terminal** (no exit) → rejected · expired · waived · completed
- **satisfies checklist** → approved · released · completed

The scenarios in this guide

Scenario	What it proves	Key commands	Section
A — Issue a House B/L	prepare-from-subject (no re-typing), checklist generate + recompute, issue → DOC-12 draft→approved, final-version lock	`prepare_from_subject`, `checklist.generate`, `issue`	§02
B — Versions, gates & legs	create/lock version + locked-write guard, per-leg document expansion (DOC-08), the DOC-13 mandatory-doc gate, waive / reject / expire	`version.create_version`, `gate.check`, `waive_item`	§03 · §04
C — Release & uploads	B/L release approval flow (release / endorse / surrender / amend), the upload → review → accept path, negative guards	`release`, `surrender`, `upload`	§05

Demo fixtures you start from (loaded by `--with-demo=all`)
On the Globex export: a draft **House B/L** `demo_doc_hbl` on the house `shipments.demo_house_001` (`origin=generated`, `status=draft`, `visibility=customer`, 3 originals; field values `bl_type=original`, `place_of_receipt=Mumbai ICD`), a draft **Shipping Instruction** `demo_doc_si` on the master, and a master **checklist** `demo_checklist_globex` (ocean / export / master) at **0.00%** with two mandatory items — **HBL** and **SI** — both still `required`.

SECTION 02 · SCENARIO A

02 Issue a House B/L — prepare, checklist, issue

Goal: take the draft Globex **House B/L** from `draft` to an issued, numbered, final-locked document that satisfies its checklist item — with parties and container numbers pulled from the subject, not re-keyed. **Role:** Documentation clerk. **Start:** `demo_doc_hbl` is `status=draft`.

#	Action	Expected result	✔ Checkpoint	Trace
1	On the draft HBL, run `prepare_from_subject`.	Parties + charges are copied from the house (`logistics.shipment.party` / `.charge`); container & seal numbers are read-through from the bound `equipment_usage_id` into field values `container_no` / `seal_no`. Additive — re-running does not duplicate.	**Return shows `parties_added` / `charges_added` / `equipment_fields_added`; nothing re-typed.**	DOC-01 BR-DOC-03
2	Run `prepare_from_subject` again on the **same** document.	No duplicate parties/charges/fields — the mapper skips a document that already carries them.	**Counts return 0 on the second run.**	BR-DOC-PARTY-02
3	Run `checklist.generate` for the master (`target_model_key=logistics.shipment`, `target_record_uuid=` the Globex master).	Additive refresh of `demo_checklist_globex` — rule-matched required documents appear; existing HBL/SI items are preserved (not re-created). A `checklist.generated` event fires.	**Checklist present; existing items untouched.**	DOC-01 BR-DOC-CHK-01/02
4	Link the HBL checklist item to this document (set `satisfied_by_document_id` → `demo_doc_hbl`), then run `issue` on the HBL.	Field schema validated; if blank, `document_number` is stamped from the `DOC` sequence; with `doc.version.lock_on_issue=true` the version is locked (`is_final=true`); a history row + status-history row are written; status flips **draft → approved**; `document.issued` fires.	**Status = approved; `is_final=true`; number populated.**	DOC-12 BR-DOC-06
5	Re-read the linked HBL checklist item and the checklist header.	Reaching a **satisfying** status (approved) auto-flips the linked item to `satisfied` (BR-DOC-STA-05); recompute lifts `completion_percent` off 0%.	**Item = satisfied; completion rises.**	BR-DOC-CHK-ITEM-02
6	Run `checklist.recompute_completion` on the master checklist.	Returns `completion_percent` = satisfied-or-waived mandatory ÷ mandatory items (SI still `required` → ~50%).	**Percent reconciles with item states.**	BR-DOC-CHK-05

✔ Scenario A pass criteria

The HBL reached **approved** (issued/final) from carried-forward subject data, got a system-composed number, locked its version, and satisfied its checklist item — zero fields re-keyed.

⚠ "Issued" is not a status

Do not look for an `issued` value — `issue` maps **draft → approved**, and `approved` IS the issued/final state (feature 05). Only a `draft` or `under_review` document can be issued — anything else returns *"only a draft/under_review document can be issued"*.

SECTION 03 · SCENARIO B

03 Versioning & the final-version lock (DOC-09)

Goal: prove corrections cut a **new version** and that a locked version can never be overwritten — correction = new version, never in-place edit. **Start:** the issued HBL from §02 (`is_final=true`).

#	Action	Expected result	✔	Trace
1	Run `version.create_version` with a `version_reason` for the HBL.	An append-only `logistics.document.version` row is cut with a JSON `field_value_snapshot` ; the document's `version_number` bumps, `current_version_id` repoints, `is_final` resets to false; `version_created` fires.	**New version row; number bumped.**	DOC-09 BR-DOC-VER-04
2	Run `version.create_version` **without** a reason (`doc.version.require_reason=true`).	Rejected by the pre-create hook: *"a version reason is required (doc.version.require_reason=true) (BR-DOC-VER-01)"*.	**Reasonless version blocked.**	BR-DOC-VER-01
3	Run `version.lock_version` against the current version row.	Version `is_locked=true` — the final-version lock is set (locks on issue/release).	**Version shows locked.**	BR-DOC-VER-02
4	Attempt to `write` (update) that **locked** version row.	Rejected by the pre-update hook: *"this version is locked (final-version lock) — create a new version instead (BR-DOC-VER-02)"*.	**Locked-version edit refused.**	BR-DOC-VER-02
5	Run `amend` on the issued document with `doc.allow_post_issue_amendment=false` (default).	Rejected: *"post-issue amendment is not permitted (doc.allow_post_issue_amendment=false) (BR-DOC-05)"*.	**Post-issue amend blocked.**	BR-DOC-05
6	Flip `doc.allow_post_issue_amendment=true` , re-run `amend` **without** a reason.	Rejected: *"a revision reason is required to amend (doc.version.require_reason=true)"*. With a reason it succeeds — bumps `revision_number` , clears `is_final` , logs history, emits `document.amended` .	**Reason enforced; amend audited.**	BR-DOC-05

Append-only version + release trails

Both `logistics.document.version` and `logistics.document.release` reject updates and deletes at the hook layer (BR-DOC-VER / BR-DOC-REL-01) — the audit trail is immutable. A released/issued version is never overwritten; a correction always cuts a new version row.

⚠ Delete is blocked past draft

The `pre.logistics.document.delete` hook refuses any document in `approved` / `released` or a terminal status: *"cannot delete a {status} document — archive or supersede instead"*. The trail must survive.

SECTION 04 · SCENARIO B — THE MULTI-LEG & GATE DEEP-DIVE

04 Per-leg documents (DOC-08) & the mandatory-doc gate (DOC-13)

Goal: prove the checklist expands **per leg** on a multimodal shipment via the leg-document matrix, and that the DOC-13 gate **blocks** a downstream action while any mandatory document is unsatisfied. **Start:** a multimodal shipment (road pre-carriage → ocean main-carriage → road delivery).

```
leg matrix : road/pickup → CMR (opt) · ocean/main_carriage → HBL·MBL·SI·VGM · road/delivery → DO·POD
           SEA→ocean · leg_type transshipment→on_carriage · expand gated by doc.checklist.expand_per_leg
```

#	Action	Expected result	✔ Checkpoint	Trace
1	With `doc.checklist.expand_per_leg=true`, run `checklist.generate` for the multimodal shipment.	Beyond the rule-matched items, one item is added **per applicable leg** from `logistics.document.leg.matrix`: ocean main-carriage pulls HBL/MBL/SI/VGM (mandatory), road delivery pulls DO/POD, road pickup pulls CMR (optional).	**Leg documents present; matrix respected.**	DOC-08
2	Run `gate.check` with `gate_code=bl_awb_release` for the subject while a mandatory item is still `required`.	Returns `passed=false`, `blocked=true`, `enforced=true`, plus a `missing_items` list of unsatisfied mandatory document types; a `document.gate_blocked` event fires.	**Gate blocks on missing mandatory doc.**	DOC-13 BR-DOC-GATE-01
3	Run `gate.check` with `gate_code=customs_filing` (seeded `is_enforced=false`) with the same gap.	`passed=false` but `blocked=false` — a non-enforced gate downgrades to a warning (`gate_warned`), not a block.	**Non-enforced gate warns, not blocks.**	BR-DOC-GATE-02
4	Run `checklist.waive_item` on a mandatory item **without** a reason.	Rejected: *"a waiver reason is required to waive a checklist item (BR-DOC-CHK-ITEM-03)"*.	**Reasonless waiver blocked.**	BR-DOC-CHK-ITEM-03
5	`waive_item` **with** a reason, then re-run `gate.check` (`bl_awb_release`).	Item → `waived`; a waived item counts as satisfying (BR-DOC-GATE-03), parent recomputes, and if it was the last gap the gate now `passes`.	**Waived item clears the gate.**	BR-DOC-GATE-03
6	On a pending document with a past `valid_until`, run `expire_due` (`doc.status.auto_expire=true`).	The sweep moves non-terminal documents past their `valid_until` to `expired` (only where the transition is legal) and stamps `expiry_check_at`.	**Expired count > 0; terminals skipped.**	BR-DOC-STA-03

⚠ Guarded transitions everywhere

Every status move runs through the guard: an illegal jump returns *"illegal status transition {from} → {to} (BR-DOC-STA-02)"*, and moving to `correction_required` / `rejected` / `waived` without a reason returns *"a reason is required to move a document to {status} (BR-DOC-STA-04)"*. Try `reject` / `waive` without a `reason` to confirm.

SECTION 05 · SCENARIO C

05 B/L release approval, surrender & uploads (guards)

Goal: drive the append-only B/L release lifecycle (release → endorse → surrender) with its approval + originals guards, then the upload → review → accept path. Half of testing this is confirming refusals. **Start:** the issued (`approved`) HBL.

#	Action	Expected result / refusal	✔	Trace
1	Run `release` with `doc.bl.release_requires_approval=true` but no approval decision.	Rejected: *"B/L release requires an approval decision (doc.bl.release_requires_approval=true) — pass release_approval_id or approved=True (BR-DOC-REL-03)"*.	**Unapproved release blocked.**	BR-DOC-REL-03
2	Re-run `release` with `approved=True` and `release_type_id=telex` .	Status **approved → released**; an append-only `logistics.document.release` row is written; `released_at` stamped; `document.released` fires.	**Released; release row appended.**	BR-DOC-REL-02
3	Run `release` passing a release type whose `applicable_to` excludes this type (e.g. `do_release` on an HBL).	Rejected: *"release type 'do_release' does not apply to document type 'HBL' (BR-DOC-REL-04)"*.	**Wrong release type refused.**	BR-DOC-REL-04
4	Run `surrender` returning fewer originals than `number_of_originals` (3), with `doc.bl.require_original_return_on_surrender=true` .	Rejected: *"originals returned (N) must match number of originals (3) on surrender (BR-DOC-REL-02)"*. Matching 3 → status **released → completed**.	**Originals count enforced.**	BR-DOC-REL-02
5	Attempt to `endorse` a document that is **not** released (still approved).	Rejected: *"only a released document can be endorsed"*. Also try to `write` a release row → *"logistics.document.release is append-only and cannot be modified"*.	**Endorse + release-edit blocked.**	BR-DOC-REL-01
6	`upload` a file → `review_upload` → `accept_upload` .	Creates `origin=uploaded` , `status=pending` , `upload_status=uploaded` ; review → `reviewed` / `under_review` ; accept → `accepted` /DOC-12 `approved` (satisfies the checklist item).	**Upload walks to accepted/approved.**	DOC-03 BR-DOC-REP-05

Upload guards worth tripping

Action	Expected refusal	Config
`upload` without a `document_type_id`	*"upload requires a document_type_id"*	—
`upload` a disallowed MIME (e.g. `exe`)	*"mime type 'exe' is not permitted (doc.upload.allowed_mime_types)"*	`doc.upload.allowed_mime_types`
`upload` a file over the size cap	*"upload size NMB exceeds the 25MB limit (doc.upload.max_size_mb)"*	`doc.upload.max_size_mb=25`
`accept_upload` before `review_upload`	*"upload must be reviewed before it can be accepted (doc.upload.require_review=true)"*	`doc.upload.require_review`

✔ Full-module pass criteria

All three scenarios green: issue + checklist satisfy (A); versioning + final lock + per-leg gate with waive/expire (B); release approval + originals-return + append-only + upload accept (C) — with every negative guard confirmed to refuse with its exact message.

Traceability: DOC- and BR-DOC-* IDs map to the logistics.documentation feature set (features 01–12) in `src/domains/logistics/docs/documentation.md` ; VGM-before-SI, per-leg matrix, and gate enforcement are config-gated (`doc.*` keys in `data/documentation_config.xml`). Cross-module effects (shipment readiness triggering auto-generation, storage byte immutability) are verified in those modules' guides.*